Apuntes de DWES

Índice de contenidos

- **Bloque 1 — Arquitecturas de desarrollo web (UT1)**
  - 1.1 Modelo cliente-servidor
  - 1.2 Arquitecturas de capas
  - 1.3 Front-end, back-end y full-stack
  - 1.4 Generación de HTML dinámico
  - 1.5 Lenguajes de programación del lado servidor
  - 1.6 El protocolo HTTP y HTTPS
  - 1.7 La evolución de la Web (1.0 → 4.0)
  - 1.8 Servidores web y entornos de trabajo
  - 1.9 Herramientas de desarrollo
  - 1.10 Glosario de la UT1
- **Bloque 2 — El lenguaje PHP (UT2)**
  - 2.1 Qué es PHP y cómo se ejecuta
  - 2.2 Insertar código PHP en HTML
  - 2.3 Ejecutar PHP: XAMPP y servidor propio
  - 2.4 Sintaxis básica
  - 2.5 Variables
  - 2.6 Constantes
  - 2.7 Tipos de datos
  - 2.8 Ámbito de las variables y superglobales
  - 2.9 Comentarios
  - 2.10 Expresiones y operadores
  - 2.11 Funciones nativas más usadas
  - 2.12 Glosario de la UT2
- **Bloque 3 — Estructuras de control, arrays y formularios (UT3)**
  - 3.1 Condicionales
  - 3.2 Bucles
  - 3.3 include / require
  - 3.4 Arrays
  - 3.5 Funciones
  - 3.6 Errores y excepciones
  - 3.7 Formularios
  - 3.8 Expresiones regulares (ampliación)
  - 3.9 Glosario de la UT3
- **Bloque 4 — Ejercicios resueltos y código explicado**
- **Bloque 5 — Glosario general**
- **Anexo A — Resumen de funciones**
- **Anexo B — Errores frecuentes y buenas prácticas**

Bloque 1 — Arquitecturas de desarrollo web (UT1)

1.1 Modelo cliente-servidor

Una **aplicación web** se ejecuta sobre dos programas que hablan entre sí:

- El **cliente** (normalmente el navegador) envía una **petición** (*request*).
- El **servidor** la procesa y devuelve una **respuesta** (*response*).

El cliente **pide** y pinta el resultado; el servidor **decide** y genera el contenido. Este reparto de tareas es la base de todo lo que veremos.

```
Navegador  --(petición HTTP)⟶  Servidor web
Navegador  ⟵(respuesta HTML)-- Servidor web
```

Idea clave: PHP es un lenguaje del **lado servidor**. El navegador nunca ve el código PHP; solo recibe el HTML "ya cocinado" que PHP ha generado.

1.2 Arquitecturas de capas

Según cómo se reparten las responsabilidades, hablamos de:

Arquitectura	Reparto	Ejemplo
1 capa	Todo se ejecuta en el cliente	Página HTML estática
2 capas	Cliente + servidor (cliente-servidor)	Programa de escritorio + BD
3 capas	Presentación / Lógica de negocio / Datos	Aplicación web típica
Multicapa	Se subdividen las tres anteriores	Grandes sistemas distribuidos

La **arquitectura de 3 capas** es la habitual en desarrollo web:

1. **Capa de presentación** (interfaz): HTML, CSS, JavaScript. Lo que ve el usuario.
2. **Capa de lógica de negocio** (aplicación): PHP, Java, C#… Procesa datos, aplica reglas.
3. **Capa de datos** (persistencia): el gestor de base de datos (MySQL, PostgreSQL…).

Idea clave: en DWES trabajamos sobre todo la **capa de lógica de negocio** (PHP) que conecta la presentación con los datos.

1.3 Front-end, back-end y full-stack

- **Front-end:** todo lo que se ejecuta en el **navegador** (HTML, CSS, JS).
- **Back-end:** todo lo que se ejecuta en el **servidor** (PHP, bases de datos…).
- **Full-stack:** quien domina las dos partes.

1.4 Generación de HTML dinámico

Una página puede ser **estática** (el servidor siempre entrega el mismo archivo) o **dinámica** (el servidor genera el HTML en el momento de la petición, según datos concretos). Para generar HTML dinámico existen varias estrategias:

- **CGI** (*Common Gateway Interface*): el servidor ejecuta un programa externo por cada petición. Sencillo pero lento (arranca un proceso cada vez).
- **Módulos del servidor** (p. ej. `mod_php` en Apache): el intérprete vive dentro del propio servidor. Es la forma habitual de ejecutar PHP con Apache.
- **Contenedores de aplicaciones** (servlets/JSP, ASP.NET…): el servidor mantiene un entorno de ejecución para las aplicaciones del lado servidor.

1.5 Lenguajes de programación del lado servidor

Algunos lenguajes clásicos del back-end: **PHP**, **JSP/Java**, **ASP.NET/C#**, **Perl**, **Python** (Django/Flask), **Ruby** (Rails), **Node.js** (JavaScript en servidor).

En este módulo trabajamos **PHP**, que destaca por:

- Estar pensado específicamente para la web.
- **Incrustarse** directamente dentro del HTML.
- Ser multiplataforma y de código abierto.

1.6 El protocolo HTTP y HTTPS

HTTP (*HyperText Transfer Protocol*) es el protocolo de la web. Detalles clave:

- Es un protocolo **sin estado** (*stateless*): cada petición es independiente; el servidor no recuerda la anterior. Para "recordar" al usuario se usan cookies y sesiones.
- Los **puertos** por defecto son **80** (HTTP) y **443** (HTTPS, cifrado con TLS).

Métodos HTTP

Método	Para qué sirve	Dónde viajan los datos
GET	Pedir un recurso / enviar datos en la URL	En la URL (`?num1=8&num2=15`)
POST	Enviar datos que modifican algo (formularios)	En el cuerpo de la petición
HEAD, PUT, DELETE…	Otros usos (REST, metadatos)	—

Códigos de estado

- **2xx** → éxito (`200 OK`).
- **3xx** → redirección (`301`, `302`).
- **4xx** → error del cliente (`404 Not Found`, `403 Forbidden`).

- **5xx** → error del servidor (`500 Internal Server Error`).

HTTPS es HTTP sobre TLS: cifra el tráfico entre cliente y servidor.

1.7 La evolución de la Web (1.0 → 4.0)

- **Web 1.0:** páginas **estáticas**, el usuario solo consulta. Contenido publicado.
- **Web 2.0:** la **web social**; el usuario genera contenido (blogs, redes, AJAX).
- **Web 3.0: web semántica**; los datos se estructuran para que las máquinas los entiendan.
- **Web 4.0:** hacia la web **ubicua / inteligente** (IoT, IA, asistentes).

1.8 Servidores web y entornos de trabajo

Un **servidor web** es el programa que atiende las peticiones HTTP y entrega las respuestas. Los más conocidos:

- **Apache HTTP Server** — el más usado con PHP; se instala en el paquete **XAMPP**.
- **Nginx** — muy rápido, habitual como *proxy* inverso.
- **IIS** — el de Microsoft.

XAMPP = **X** (multiplataforma) + **A**pache + **M**ariaDB/MySQL + **P**HP + **P**erl. Es el entorno todo-en-uno que usamos para levantar un servidor local de desarrollo.

1.9 Herramientas de desarrollo

- **Editors** ligeros: VS Code, Sublime Text, Notepad++.
- **IDE** (entorno integrado): NetBeans, PhpStorm, Eclipse. Añaden depuración, autocompletado, gestión de proyectos, integración con el servidor, etc.

1.10 Glosario de la UT1

Término	Significado
Cliente	Programa que solicita un recurso (el navegador).
Servidor	Programa que atiende peticiones y genera respuestas.
HTTP / HTTPS	Protocolo de transferencia de la web (HTTPS cifrado).
GET / POST	Métodos para enviar datos al servidor.
Front-end	Parte que se ejecuta en el navegador.
Back-end	Parte que se ejecuta en el servidor.
CGI	Interfaz para ejecutar programas externos desde el servidor.
XAMPP	Paquete Apache + MySQL + PHP + Perl.
Petición (request)	Mensaje que el cliente envía al servidor.
Respuesta (response)	Mensaje que el servidor devuelve al cliente.

Término	Significado
Stateless	Sin estado: el servidor no recuerda peticiones anteriores.

Bloque 2 — El lenguaje PHP (UT2)

2.1 Qué es PHP y cómo se ejecuta

PHP (*PHP: Hypertext Preprocessor*) es un lenguaje de programación **del lado del servidor** y de **propósito general**, aunque nació para la web. Su ideología es **incrustarse dentro del HTML**: el fichero es HTML y, en los puntos que interese, se abren "islas" de PHP.

El ciclo es siempre el mismo:

1. El navegador pide `pagina.php`.
2. El servidor (Apache) detecta la extensión `.php` y **lo ejecuta con el intérprete**.
3. El intérprete **sustituye el código PHP por su resultado** (normalmente texto/HTML).
4. El servidor entrega al navegador **solo el HTML resultante**. El código fuente PHP nunca sale del servidor.

Un fichero `.php` puede contener **HTML, CSS, JavaScript y PHP mezclados**.

2.2 Insertar código PHP en HTML

Se abre con `<?php` y se cierra con `?>`. Todo lo que hay fuera de esas etiquetas se envía tal cual al navegador.

```
<!DOCTYPE html>
<html>
<body>
  <h1>Mi primera página</h1>
  <?php
    echo "<p>Hola desde PHP</p>";
  ?>
</body>
</html>
```

Para **imprimir un valor** de forma abreviada existe la forma corta `<?= ... ?>`, que equivale a `<?php echo ... ?>`:

```
<p>La suma es <?= 2 + 3 ?></p>
```

2.3 Ejecutar PHP: XAMPP y servidor propio

Hay dos formas habituales de probar un `.php`:

a) Con XAMPP (Apache). Se coloca el proyecto dentro de `htdocs` y se accede por `http://localhost/carpeta/fichero.php`.

b) Con el servidor propio de PHP (el que pide el enunciado del ejercicio 5 de UT2). Desde la carpeta del proyecto:

```
php -S localhost:8000
```

Y se abre `http://localhost:8000/fichero.php`. Este servidor es ideal para pruebas rápidas porque no hay que configurar nada.

Ejemplo (de mis ejercicios): el Ejercicio 5 de UT2 se ejecuta con `php -S localhost:8000` situándose en la carpeta `Ejercicio5`.

2.4 Sintaxis básica

- Cada **sentencia** termina en **punto y coma** `;`.
- El **nombre de variables** empieza por `$`.
- Los **nombres** distinguen mayúsculas de minúsculas (`$edad` y `$Edad` son distintas), pero **las palabras clave** (`if`, `echo`, `function`) no.
- **Comentarios**: `//` o `#` para una línea, `/* ... */` para varias.

```php
<?php
$nombre = "Ana";   // una variable
echo "Hola, $nombre";  // echo imprime
```

2.5 Variables

Una **variable** es un contenedor con un nombre que guarda un valor. En PHP:

- Se declaran **al usarlas**, al asignarles un valor con `=`.
- **No se declara el tipo**: PHP decide el tipo según el valor (tipado débil/dinámico).
- El **tipo puede cambiar** a lo largo del programa.

```php
$edad = 18;          // entero
$edad = "dieciocho"; // ahora es una cadena (PHP lo permite)
```

Asignación por copia

Por defecto, `$b = $a` **copia el valor**.

```php
$a = 5;
$b = $a;   // $b recibe una copia del 5
$b = 9;    // cambiar $b NO afecta a $a
echo $a;   // 5
```

Asignación por referencia

Con `&` se asigna **la referencia**: las dos variables apuntan al **mismo** valor.

```php
$a = 5;
$b = &$a;  // $b es un alias de $a
$b = 9;    // al cambiar $b, cambia $a
echo $a;   // 9
```

Variables no inicializadas

Si se usa una variable a la que no se le ha dado valor, PHP lanza un aviso (`E_WARNING` / `E_NOTICE`) y la trata como `null`. Por eso conviene inicializarlas.

Variables variables

El nombre de una variable puede venir de otra variable usando `$$`.

```php
$nombre = "saludo";
$$nombre = "Hola";  // crea $saludo = "Hola"
echo $saludo;       // Hola
```

Idea clave: cuando lees del formulario `$numero = intval($_GET['numero'] ?? 0)`, el `?? 0` evita el aviso de "variable no definida" si no llega el parámetro.

2.6 Constantes

Una **constante** guarda un valor que **no cambia** durante la ejecución. Se definen con `define()` y, por convención, se escriben **en mayúsculas**. Se usan **sin** `$`.

```php
define("PI", 3.141592);
echo PI;   // se usa sin el signo dollar
```

PHP ya trae muchas constantes predefinidas:

Constante	Valor
`PHP_VERSION`	Versión de PHP en ejecución
`PHP_INT_MAX`	Mayor entero que admite la máquina
`PHP_INT_SIZE`	Tamaño en bytes de un entero
`PHP_EOL`	Salto de línea del sistema
`PHP_OS`	Sistema operativo

También existen las llamadas **constantes mágicas** (dependen de dónde se usan): `__LINE__`, `__FILE__`, `__DIR__`, `__FUNCTION__`.

2.7 Tipos de datos

PHP maneja **ocho tipos** primitivos, agrupados así:

Grupo	Tipos
Escalares	`int` (entero), `float` (coma flotante), `bool` (lógico), `string` (cadena)
Compuestos	`array` (matriz), `object` (objeto)
Especiales	`resource` (recurso), `null`

Enteros (int)

```
$entero = 42;
$negativo = -7;
```

Coma flotante (float / double)

Con parte decimal. El separador decimal en el código es el **punto**.

```
$precio = 19.95;
```

Lógicos (bool)

Solo dos valores: `true` o `false` (se escriben sin comillas, en minúscula).

Reglas de conversión a booleano (muy preguntadas):

- `false` es: `false`, `0`, `0.0`, `""`, `"0"`, `[]`, `null`.
- Todo lo demás es `true`.

```
var_dump((bool) 0);      // false
var_dump((bool) "0");    // false
var_dump((bool) "0.0");  // true  (¡no está en la lista!)
var_dump((bool) []);     // false
```

Cadenas (string)

Comillas **dobles** → PHP **interpola** las variables ($var se sustituye por su valor). Comillas **simples** → texto **literal**, no interpola.

```
$nombre = "Ana";
echo "Hola, $nombre";  // Hola, Ana
echo 'Hola, $nombre';  // Hola, $nombre
```

Función	Para qué
`strlen($s)`	Longitud en **bytes**
`mb_strlen($s)`	Longitud en **caracteres** (usa esto si hay tildes)
`strtoupper` / `strtolower`	Mayúsculas/minúsculas (bytes)
`mb_strtoupper` / `mb_strtolower`	Mayúsculas/minúsculas (multibyte, con tildes)
`ucfirst($s)`	Primera letra en mayúscula
`ucwords($s)`	Primera letra de **cada palabra** en mayúscula
`substr` / `mb_substr`	Extraer un trozo de cadena
`str_replace($busc,$p,$s)`	Reemplazar `$busc` por `$p` en `$s`
`trim` / `ltrim` / `rtrim`	Quitar espacios (ambos lados / izq. / der.)
`substr_count($s,$sub)`	Contar cuántas veces aparece `$sub` en `$s`
`strpos($s,$sub)`	Posición de `$sub` (o `false` si no está)
`explode($sep,$s)`	Convertir cadena → array
`implode($sep,$arr)`	Unir array → cadena

Nulos (null)

`null` es el valor de una variable **sin valor**. Se comprueba con `is_null()`.

Arrays y objetos

Los veremos con detalle en el bloque 3 (`array`) y más adelante (`object`).

2.8 Ámbito de las variables y superglobales

Ámbito (scope)

- Una variable **nueva** dentro de una función es **local** (muere al salir).
- Las variables de **fuera no** se ven dentro de una función salvo que se declaren con `global` o se acceda con `$GLOBALS`.
- Una variable **nueva** creada dentro de un `include` es **global** (vive en el ámbito donde se incluye el fichero).

```php
$contador = 0;
function incrementar() {
  global $contador;   // así se accede a la de fuera
  $contador++;
}
```

Superglobales (variables predefinidas)

Son **arrays** que PHP rellena automáticamente y que están **disponibles en todo** el script (dentro y fuera de funciones):

Superglobal	Contenido
`$_SERVER`	Datos del servidor y de la petición (`SERVER_NAME` , `REQUEST_METHOD` , `PHP_SELF` ...)
`$_GET`	Parámetros enviados por **GET**
`$_POST`	Datos enviados por **POST**
`$_REQUEST`	GET + POST + cookies
`$_FILES`	Ficheros subidos
`$_COOKIE`	Cookies
`$_SESSION`	Variables de sesión
`$_ENV`	Variables de entorno
`$GLOBALS`	Todas las variables globales

```php
// Unos ejemplos de $_SERVER
echo $_SERVER['SERVER_NAME'];     // nombre del servidor
echo $_SERVER['SERVER_SOFTWARE']; // p. ej. Apache/2.4
echo $_SERVER['REQUEST_METHOD'];  // GET o POST
```

2.9 Comentarios

```php
<?php
// Comentario de una línea
# Otro comentario de una línea
/*
  Comentario
  de varias líneas
*/
```

2.10 Expresiones y operadores

Una **expresión** es todo lo que produce un valor (`2 + 3` , `$a . $b` ...).

Asignación

```php
$a = 5;
$a += 3;   // $a = $a + 3   → 8
$a -= 2;   // → 6
$a *= 2;   // → 12
$s = "Hola";
$s .= " mundo";  // concatena → "Hola mundo"
```

Aritméticos

Operador	Significado
`+ - * /`	Suma, resta, multiplicación, división
`%`	**Resto** (módulo)
`**`	Potencia (`2 ** 3` = 8)

Idea clave: `%` es muy útil; p. ej. `$i % 2 == 0` detecta números **pares**.

Comparación

Operador	Significado
`==`	Igual (compara **valor**, con conversión de tipos)
`===`	**Idéntico** (mismo valor **y mismo tipo**)
`≠` / `<>`	Distinto
`≠=`	No idéntico
`< > ≤ ≥`	Menor, mayor, menor o igual, mayor o igual
`⟺`	Nave espacial: -1, 0, 1
`??`	*Null coalescing*: usa el valor si existe y no es null

Lógicos

`and` / `&&` (y), `or` / `||` (o), `xor` (o exclusivo), `!` (negación).

Precedencia: `!` > `&&` / `and` > `||` / `or`. Ante la duda, **paréntesis**.

Incremento / decremento

```php
$i++;  // incrementa después de usar
++$i;  // incrementa antes de usar
$i--;  // decrementa
```

Concatenación

El punto `.` une cadenas: `"Hola " . "mundo"`.

2.11 Funciones nativas más usadas

Numéricas

Función	Para qué
`abs($x)`	Valor absoluto
`round($x, $n)`	Redondea a `$n` decimales
`floor($x)` / `ceil($x)`	Redondea abajo / arriba
`max(...)` / `min(...)`	Mayor / menor de una lista
`rand($min,$max)`	Número **seudoaleatorio**
`random_int($min,$max)`	Entero aleatorio **criptográficamente seguro** (desde PHP 7)
`intval($x)` / `floatval($x)`	Convertir a entero / a flotante

Ejemplo: `rand(1, 1000)` devuelve un entero al azar entre 1 y 1000.

De cadenas

Ver la tabla del apartado 2.7. Añadimos las de **formato de salida**:

Función	Para qué
`var_dump($v)`	Muestra **tipo y valor** (para depurar)
`print_r($v)`	Muestra el valor de forma legible (arrays)
`htmlspecialchars($s)`	Convierte `< > & "` en entidades (seguridad)
`str_contains($s,$sub)`	¿Contiene la subcadena? (PHP 8)
`strtr($s,$map)`	Traduce caracteres según un array

De tipos (comprobación y conversión)

`is_int()`, `is_float()`, `is_bool()`, `is_string()`, `is_array()`, `is_null()`, `is_numeric()`, `isset($v)` (existe y no es null), `empty($v)` (no existe o es "vacío"), `settype($v,"tipo")`.

Ejemplo (de mis ejercicios):

```php
$numFloat = 12;          // es un int, no float
is_float($numFloat);     // false
```

De fecha y hora

Función	Para qué
`date($formato)`	Fecha/hora actual formateada
`time()`	Marca de tiempo actual (segundos desde 1970)
`mktime( ... )`	Marca de tiempo de una fecha concreta
`strtotime($str)`	Convierte texto de fecha a marca de tiempo
`checkdate($m,$d,$a)`	¿Es una fecha válida?

Caracteres de formato de `date()`:

Carácter	Significa	Ejemplo
`d`	Día con 2 dígitos	05
`j`	Día sin ceros	5
`m`	Mes con 2 dígitos	03
`n`	Mes sin ceros	3
`Y`	Año con 4 dígitos	2026
`l` (ele)	Día de la semana	Monday
`F`	Mes en letra	March
`H:i:s`	Hora:minuto:segundo (24 h)	14:07:03

Ejemplo: `date("l, F jS Y")` → "Monday, March 5th 2026" (formato inglés por defecto). `date("d/m/Y H:i")` → "05/03/2026 14:07".

2.12 Glosario de la UT2

Término	Significado
PHP	Lenguaje de programación del lado servidor.
Interpolar	Sustituir `$var` por su valor dentro de comillas dobles.
Ámbito (scope)	Zona donde una variable existe y es accesible.
Superglobal	Array predefinido accesible desde cualquier punto.
Constante	Valor inmutable definido con `define()` y usado sin `$`.
Tipado débil	El tipo lo decide el valor, no se declara.
mb_*	Funciones que trabajan con caracteres (UTF-8) en vez de bytes.
??	Operador que usa un valor por defecto si no existe o es null.

Bloque 3 — Estructuras de control, arrays y formularios (UT3)

3.1 Condicionales

if / else / elseif

```php
if ($nota ≥ 5) {
    echo "Aprobado";
} elseif ($nota ≥ 9) {
    echo "Sobresaliente";
} else {
    echo "Suspenso";
}
```

La misma clasificación, escribiendo las condiciones de menor a mayor:

```php
if ($nota < 5) {
    $cat = "Insuficiente";
} elseif ($nota < 6) {
    $cat = "Suficiente";
} elseif ($nota < 7) {
    $cat = "Bien";
} elseif ($nota < 9) {
    $cat = "Notable";
} else {                // 9 y 10
    $cat = "Sobresaliente";
}
```

Idea clave: los intervalos con corchetes `[0,5)` significan "0 incluido, 5 no incluido"; por eso se escribe `< 5` y no `≤ 5`.

switch

Se usa cuando se compara **la misma variable** contra varios valores concretos.

```php
switch ($dia) {
    case 1:
        echo "Lunes";
        break;
    case 2:
        echo "Martes";
        break;
    default:
        echo "Otro día";
}
```

- El `break` **corta** el `switch`. Sin él, PHP "cae" en el `case` siguiente (*fall-through*), ejecutando código de más.
- `default` recoge los casos no contemplados.

Operador ternario

Forma abreviada de if/else: `condicion ? valorSiTrue : valorSiFalse`.

```php
$estado = ($nota ≥ 5) ? "Aprobado" : "Suspenso";
$texto  = ($veces == 1) ? "vez" : "veces";   // singular/plural
```

3.2 Bucles

while

Repite **mientras** se cumpla la condición (puede ejecutarse **0 veces**).

```php
$i = 1;
while ($i ≤ 10) {
    echo $i;
    $i++;
}
```

do...while

Como `while`, pero **se ejecuta al menos una vez** (la condición se comprueba al final).

```php
$i = 1;
do {
    echo $i;
    $i++;
} while ($i ≤ 10);
```

for

Ideal cuando se conoce el **número de repeticiones**.

```php
for ($i = 1; $i ≤ 10; $i++) {
    echo $i;
}
```

Estructura: `for (inicio; condición; incremento)`.

Truco del módulo: para recorrer solo los **pares**, en el incremento se avanza de 2 en 2 (`$i += 2`) o se comprueba `$i % 2 == 0`.

foreach

Recorre **arrays** de forma sencilla.

```php
foreach ($frutas as $fruta) {
    echo $fruta;
}

foreach ($frutas as $posicion ⇒ $fruta) {   // con la clave
    echo "$posicion: $fruta";
}
```

break y continue

- `break` **sale** del bucle.
- `continue` **salta** a la siguiente vuelta (omite el resto del cuerpo en esa iteración).

```php
for ($i = 1; $i ≤ 10; $i++) {
    if ($i == 5) continue;   // salta el 5
    if ($i == 8) break;      // se detiene en el 8
    echo $i;
}
```

3.3 include / require

Permiten **insertar** el contenido de otro fichero (como si se copiara ahí).

Sentencia	Si falta el fichero
`include`	Aviso (`warning`) y **sigue**
`require`	**Error fatal** y **para**
`include_once` / `require_once`	Igual, pero solo lo incluye una vez

```php
include 'ejercicios/ejercicio1.php';
require_once 'funciones.php';
```

Ejemplo (de mis ejercicios): en `UT3_1` el `index.php` incluye cada ejercicio con `<?php include 'ejercicios/ejercicioN.php'; ?>`, así el index queda limpio.

return

Dentro de una función, `return` devuelve un valor y **termina** la función. En un fichero incluido, `return` termina la ejecución de ese fichero incluido.

3.4 Arrays

Un **array** es una variable que guarda **varios valores** en una sola estructura.

Tipos

- **Indexados:** claves numéricas 0, 1, 2…
- **Asociativos:** claves de texto (`'nombre' ⇒ 'Ana'`).
- **Multidimensionales (matrices):** arrays dentro de arrays.

Crear un array

```php
$frutas = array('fresa', 'naranja', 'uva');
$frutas = ['fresa', 'naranja', 'uva'];    // sintaxis corta (recomendada)

$persona = [
    'nombre' ⇒ 'Ana',
    'curso'  ⇒ '2º DAW',
];
```

Acceder y modificar

```php
echo $frutas[0];          // fresa
$frutas[0] = 'manzana';   // modificar
$frutas[] = 'pera';       // añadir al final (clave automática)
echo $persona['nombre'];  // Ana
```

Funciones útiles con arrays

Función	Para qué
`count($a)`	Número de elementos
`range($a,$b)`	Array con los valores de `$a` a `$b`
`array_rand($a)`	Clave **aleatoria** del array

Función	Para qué
`array_merge($a,$b)`	Unir arrays
`sort` / `rsort`	Ordenar ascendente / descendente
`in_array($v,$a)`	¿Está `$v` dentro de `$a`?
`array_keys` / `array_values`	Obtener claves / valores
`implode($sep,$a)`	Array → cadena
`explode($sep,$s)`	Cadena → array
`str_split($s)`	Cadena → array de caracteres

Recorrer un array

```php
foreach ($frutas as $fruta) {          // solo valores
    echo $fruta;
}

foreach ($frutas as $i => $fruta) {    // clave y valor
    echo ($i + 1) . ": $fruta";
}
```

Con `for` (para arrays indexados):

```php
for ($i = 0; $i < count($frutas); $i++) {
    echo $frutas[$i];
}
```

Matrices (arrays multidimensionales)

```php
$notas = [
    'Ana'   => [7, 8, 6],
    'Luis'  => [5, 9, 10],
];

echo $notas['Ana'][0];   // 7  (fila 'Ana', primera nota)

foreach ($notas as $alumno => $susNotas) {
    foreach ($susNotas as $nota) {
        echo "$alumno: $nota";
    }
}
```

Una cadena es como un array de caracteres

```php
$s = "Hola";
echo $s[0];     // H
echo $s[1];     // o
```

El operador spread `...`

Con PHP 7.4+ se puede extender un array dentro de otro:

```php
$a = [1, 2, 3];
$b = [0, ...$a, 4];   // [0, 1, 2, 3, 4]
```

Operadores de comparación de arrays

- `+` **unión**: añade los elementos que no estén en el primero.
- `==` **igual**: mismos pares clave/valor (sin importar orden).
- `===` **idéntico**: mismos pares **en el mismo orden y del mismo tipo**.

Idea clave: en los arrays **asociativos**, `foreach` con `clave ⇒ valor` es la forma natural de recorrerlos.

3.5 Funciones

Definir y usar

```php
function suma($a, $b) {
    return $a + $b;
}

echo suma(2, 3);   // 5
```

Argumentos: por valor y por referencia

- Por defecto, los argumentos se pasan **por valor** (se copian).
- Con `&` se pasan **por referencia** (la función modifica la original).

```php
function duplicar($n)   { $n *= 2; }        // NO cambia el original
function duplicar2(&$n) { $n *= 2; }        // SÍ cambia el original

$x = 5; duplicar($x);   echo $x;  // 5
$y = 5; duplicar2($y);  echo $y;  // 10
```

Valores por defecto

```php
function saludar($nombre = "mundo") {
    return "Hola, $nombre";
}
echo saludar();          // Hola, mundo
echo saludar("Ana");  // Hola, Ana
```

Funciones como argumentos (callbacks)

Una función puede recibir **otra función** como parámetro:

```php
$doble = fn($n) => $n * 2;   // función flecha (PHP 7.4+)

function aplicar($lista, $fn) {
    $res = [];
    foreach ($lista as $x) {
        $res[] = $fn($x);
    }
    return $res;
}

print_r(aplicar([1, 2, 3], $doble));   // 2, 4, 6
```

3.6 Errores y excepciones

Niveles de error

Constante	Gravedad
`E_ERROR`	Error fatal; detiene el script
`E_WARNING`	Aviso; el script continúa
`E_NOTICE`	Observación (variable no definida...)
`E_DEPRECATED`	Uso de funciones obsoletas
`E_ALL`	Todos

Directivas de configuración

- `error_reporting` → qué errores se muestran.
- `display_errors` → si se muestran por pantalla (en **desarrollo** sí; en producción no).
- `log_errors` / `error_log` → guardar los errores en un fichero.

Excepciones: try / catch / finally

```php
try {
    if ($dato < 0) {
        throw new Exception("El dato no puede ser negativo");
    }
    echo "Todo bien";
} catch (Exception $e) {
    echo "Error: " . $e->getMessage();
} finally {
    echo "Esto se ejecuta siempre";
}
```

- `throw` **lanza** una excepción.
- `catch` la **captura** y actúa.
- `finally` se ejecuta **pase lo que pase**.

3.7 Formularios

La etiqueta

```html
<form action="procesar.php" method="post">
    <label for="nombre">Nombre:</label>
    <input type="text" id="nombre" name="nombre" required>
    <input type="submit" value="Enviar">
</form>
```

- `action`: página que procesa los datos (por defecto, la misma).
- `method`: `get` o `post`.
- Cada control debe tener un `name`: es la clave con la que llega al servidor.

Recibir los datos

Método	Se lee de...	Dónde viaja
GET	`$_GET['name']`	URL
POST	`$_POST['name']`	Cuerpo

```php
$nombre = $_POST['nombre'] ?? '';
```

Validar y asegurar

```php
$nombre = trim($_POST['nombre'] ?? '');
$nombre = htmlspecialchars($nombre);          // evita inyección de HTML/JS
$numero = intval($_POST['numero'] ?? 0);      // convierte a número
$email  = filter_input(INPUT_POST, 'email', FILTER_VALIDATE_EMAIL);
```

Un formulario que se procesa a sí mismo

Es habitual poner el formulario y su procesamiento en el **mismo fichero**:

```php
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    // ... procesar los datos ...
}
```

Así se comprueba el método de la petición: si es POST, es que se envió el formulario.

```php
<!— También se puede usar $_SERVER['PHP_SELF'] como action —>
<form action="<?php echo htmlspecialchars($_SERVER['PHP_SELF']); ?>" method="post">
```

Redirección tras procesar

Muy usado en el patrón POST-Redirección-GET para evitar reenvíos al refrescar:

```php
header("Location: bienvenida.php");
exit;
```

3.8 Expresiones regulares (ampliación)

Una **expresión regular** (regex) es un patrón para buscar/validar texto. En PHP se usan sobre todo las funciones `preg_*`.

Metacaracteres básicos

Patrón	Significa
`.`	Un carácter cualquiera
`*`	0 o más veces
`+`	1 o más veces
`?`	0 o 1 vez (opcional)
`^` / `$`	Inicio / fin de la cadena
`[abc]`	Uno de esos caracteres
`[^abc]`	Cualquiera menos esos

Patrón	Significa
`[0-9]`, `\d`	Un dígito
`\w`	Letra, dígito o guion bajo
`\s`	Espacio en blanco
`{n,m}`	Entre n y m veces
`(...)`	Grupo

Los delimitadores más usados en PHP son las barras: `"/patron/"`.

Funciones principales

```php
preg_match("/^[0-9]{5}$/", $cp);        // ¿coincide? → 1 o 0
preg_match_all("/\d+/", $texto, $m);    // todas las coincidencias → $m
preg_replace("/\s+/", " ", $texto);      // sustituir
preg_split("/[,;]/", $lista);            // dividir por , o ;
```

Ejemplo: validar un teléfono de 9 dígitos → `preg_match("/^[0-9]{9}$/", $tel)`.

3.9 Glosario de la UT3

Término	Significado
Condicional	Estructura que decide qué camino tomar (`if`, `switch`).
Bucle	Estructura que repite código (`for`, `while`, `foreach`).
Array indexado	Array con claves numéricas.
Array asociativo	Array con claves de texto.
Matriz	Array multidimensional (array de arrays).
Foreach	Bucle para recorrer arrays.
include / require	Insertar el contenido de otro fichero.
Callback	Función que se pasa como argumento a otra.
Excepción	Error controlable que se lanza con `throw` y se captura con `catch`.
Superglobal `$_POST`	Datos enviados por POST.
Sanitizar	Limpiar/escapar los datos que llegan del usuario.

Bloque 4 — Ejercicios resueltos y código explicado

En este bloque despedazo el **código real** de los ejercicios. Para cada uno: el **enunciado** (resumido), el **código comentado** y una explicación de **qué resuelve** y **qué funciones usa**.

4.1 Ejemplo base: `básico.php`

Enunciado / objetivo: estructura mínima de una página PHP con HTML5, Bootstrap y una pequeña parte dinámica generada en el servidor.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Ejercicios de PHP - Entorno Servidor</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
        rel="stylesheet">
</head>
<body>
  <main class="container my-5">
    <header class="mb-4 text-center">
      <h1 class="display-5">Desarrollo Web en Entorno Servidor</h1>
      <p class="text-muted">Estructura básica HTML5 con PHP y Bootstrap</p>
    </header>

    <div class="card shadow-sm">
      <div class="card-body">
        <?php
          $mensaje = "¡Hola Mundo! Tu servidor PHP está funcionando correctamente.";
          $fechaActual = date("d/m/Y H:i");
          echo "<h2 class='h4 text-primary'>Mensaje desde el servidor:</h2>";
          echo "<p class='lead'>$mensaje</p>";
          echo "<hr>";
          echo "<p class='text-secondary small'>Fecha y hora de generación:
        $fechaActual</p>";
        ?>
      </div>
    </div>
  </main>

  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js">
        </script>
</body>
</html>
```

Desglose:

- `<!DOCTYPE html>` … `<link …>` → **esqueleto HTML5**. El `<meta charset="UTF-8">` es clave: sin él las tildes y la `ñ` se ven mal. Bootstrap se carga **desde un CDN**.
- `<?php … ?>` → **isla de PHP** dentro del HTML. Aquí está toda la lógica.
- `$mensaje = "…"` → variable de tipo cadena.
- `date("d/m/Y H:i")` → devuelve la fecha/hora **del servidor** en el formato indicado.
- `echo "<p class='lead'>$mensaje</p>";` → imprime HTML **interpolando** la variable (por eso comillas dobles). Dentro, los atributos del HTML van con **comillas simples** para no chocar con las dobles de `echo`.

Qué resuelve: comprobar que el servidor interpreta PHP y genera HTML dinámico (la fecha cambia en cada recarga).

Funciones usadas: `date()`.

4.2 UT2_1 — Ejercicios iniciales (opcionales)

Los tres ejercicios se muestran en **pestañas** (`nav-tabs` de Bootstrap) dentro de un único `index.php`. El JS de Bootstrap lee `data-bs-toggle="tab"` y `data-bs-target="#panel"`.

Ejercicio 1 — Número aleatorio con tamaño aleatorio

Enunciado: mostrar un número aleatorio entre 1 y 1000, con un tamaño de letra también aleatorio entre el 200 % y el 800 %.

```php
$numero = rand(1, 1000);
$tamano = rand(200, 800);
echo "<h2 class='h4 text-primary'>Ejercicio 1: número aleatorio</h2>";
echo "<p style='font-size: $tamano%;'>$numero</p>";
echo "<hr>";
echo "<p class='text-secondary small'>Tamaño de la letra: $tamano%</p>";
```

Desglose: `rand(1,1000)` genera el número; `rand(200,800)` el porcentaje de tamaño. El `style='font-size: $tamano%;'` inyecta el valor en el CSS. La interpolación `$tamano` funciona porque `echo` usa comillas dobles.

Qué resuelve: generar contenido y estilo **distintos en cada recarga**.

Funciones usadas: `rand()`.

Ejercicio 2 — Emoticono Unicode aleatorio

Enunciado: mostrar un emoticono al azar entre los códigos Unicode 128512 y 128586.

```php
$emoticono = rand(128512, 128586);
echo "<h2 class='h4 text-primary'>Ejercicio 2: emoticono Unicode</h2>";
echo "<p style='font-size: 100px;'>&#$emoticono;</p>";
echo "<hr>";
echo "<p class='text-secondary small'>Código Unicode: $emoticono</p>";
```

Desglose: HTML permite escribir un carácter por su **código Unicode** con la forma `&#codigo;`. Como el código lo genera `rand()`, se escribe `&#` + `$emoticono` + `;`.

Qué resuelve: mostrar símbolos que no se pueden teclear directamente.

Funciones usadas: `rand()`.

Ejercicio 3 — Contar la letra “t”

Enunciado: contar cuántas “t” minúsculas hay en `"This is a test"` y explicar cómo contar también las mayúsculas.

```php
$frase = "This is a test";
$minusculas = substr_count($frase, "t");
// substr_count() distingue mayúsculas; sumo las "T" aparte
$todas = $minusculas + substr_count($frase, "T");

echo "<p>1) Número de 't' minúsculas: <strong>$minusculas</strong></p>";
echo "<p>2) Número de todas las 't': <strong>$todas</strong></p>";
```

Desglose: `substr_count($texto, $sub)` cuenta apariciones y **distingue mayúsculas**, por eso para el apartado 2 se suma otra cuenta con `"T"`.

Qué resuelve: contar subcadenas dentro de un texto.

Funciones usadas: `substr_count()`.

4.3 UT2_2 — Ejercicios para practicar (opcionales)

Cinco ejercicios en pestañas.

Ejercicio 1 — Fecha y hora actual

```php
echo "<p>La fecha de hoy es: <strong>" . date("l, F jS Y") . "</strong></p>";
echo "<p>Y la hora es: <strong>" . date("H:i:s") . "</strong></p>";
```

Desglose: `date()` con distintos formatos. El punto `.` **concatena** el texto fijo con el resultado de `date()`. `l, F jS Y` da día de la semana, mes, día con ordinal y año.

Funciones usadas: `date()`.

Ejercicio 2 — Operaciones con dos números (8 y 5)

```php
$primerNumero = 8;
$segundoNumero = 5;

$resto    = $primerNumero % 5;              // %
$division = $primerNumero / $segundoNumero; // /
$suma     = $primerNumero + $segundoNumero; // +

$divisionRedondeada = round($division, 1);              // 1.6
$divisionBonita = str_replace(".", ",", $divisionRedondeada); // 1,6
```

Desglose: `%` da el **resto**; `/` la división; `+` la suma. `round($x, 1)` redondea a 1 decimal y `str_replace(".", ",", ...)` cambia el punto por coma para mostrarlo a la española.

Funciones usadas: `round()`, `str_replace()`.

Ejercicio 3 — Quitar espacios de una frase (y contar bien con tildes)

```php
$frase = "Ya decidí que esta noche se sale con todas mis motomami ";
$fraseSinEspacios = str_replace(" ", "", $frase);

$longitudOriginal    = mb_strlen($frase);
$longitudFinal       = mb_strlen($fraseSinEspacios);
$longitudConStrlen   = strlen($frase);   // da un número mayor por la "í"
```

Desglose: `str_replace(" ", "", $frase)` **quita todos los espacios** (los sustituye por cadena vacía). `mb_strlen()` cuenta **caracteres**; `strlen()` cuenta **bytes**, así que da un número mayor por la `í` (que en UTF-8 ocupa 2 bytes).

Qué resuelve: limpiar texto y demuestra por qué hay que usar las funciones `mb_*`.

Funciones usadas: `str_replace()`, `mb_strlen()`, `strlen()`.

Ejercicio 4 — Constantes (PI y máximo entero)

```php
define("PI", 3.141592);

$radio = 2;
$area = PI * $radio * $radio;
$perimetro = 2 * PI * $radio;

$maximoEntero = PHP_INT_MAX;

$areaBonita = str_replace(".", ",", round($area, 6));
echo "<p>El valor máximo que puede tomar un entero en PHP es <strong>$maximoEntero</strong>
        </p>";
```

Desglose: `define("PI", ...)` crea la **constante** sin `$`. Se usa como un número más. `PHP_INT_MAX` es una **constante predefinida** de PHP. Se redondea y se cambia el punto por coma igual que antes.

Funciones usadas: `define()`, `round()`, `str_replace()`; constante `PHP_INT_MAX`.

Ejercicio 5 — Tipos de dato (con `php -S`)

Enunciado: en un proyecto aparte, con `php -S localhost:8000`, comprobar si `$numFloat` es float y si una variable sin valor es `null`. Luego repetir cambiando **solo los valores**.

```php
$numFloat = 5.7;
if (is_float($numFloat)) {
    echo "<p>La variable <strong>numFloat</strong> vale <strong>$numFloat</strong> y
        <strong>sí es float</strong>.</p>";
} else {
    echo "<p>La variable <strong>numFloat</strong> vale <strong>$numFloat</strong> y
        <strong>no es float</strong>.</p>";
}

$variableSinValor = null;
if (is_null($variableSinValor)) {
    echo "<p>La variable <strong>variableSinValor</strong> <strong>sí es NULL</strong>.</p>";
} else {
    echo "<p>La variable <strong>variableSinValor</strong> <strong>no es NULL</strong>.</p>";
}
```

En la **salida 2** (`tiposDeDato2.php`) solo cambian los valores:

```php
$numFloat = 12;                          // ahora es int → is_float() = false
$variableSinValor = "Ahora ya tengo un valor";  // ya no es null
```

Desglose: `is_float()` y `is_null()` **comprueban el tipo** y devuelven `true`/`false`, que se usan como condición del `if`. La gracia del ejercicio es ver que el **mismo código** cambia de resultado según el **valor** de la variable.

Funciones usadas: `is_float()`, `is_null()`.

4.4 UT2_3 — Ejercicio avanzado (obligatorio)

Tres páginas enlazadas desde un `index.php`. Los datos viajan por **GET** (en la URL).

index.php — Página principal con enlaces

```php
<a class="btn btn-primary btn-lg"
   href="matematicas.php?num1=8&amp;num2=15&amp;num3=3">Operaciones Matemáticas</a>

<a class="btn btn-success btn-lg"
   href="cadena.php?cadena1=Paco tiene una carniceria&amp;cadena2=en la esquina de la
        calle">Operaciones con Cadenas</a>

<a class="btn btn-outline-primary btn-lg" href="infoServidor.php">Información del
        servidor</a>
```

Desglose: el `?` en el `href` inicia los **parámetros GET**; cada par es `clave=valor` separado por `&` (que en HTML válido se escribe `&`).

matematicas.php — leer GET y calcular

```php
$num1 = intval($_GET['num1']);
$num2 = intval($_GET['num2']);
$num3 = intval($_GET['num3']);

$suma     = $num1 + $num2 + $num3;
$producto = $num1 * $num2 * $num3;
$media    = round(($num1 + $num2 + $num3) / 3, 2);

// mayor y menor a mano, con if/elseif
if ($num1 > $num2 && $num1 > $num3) {
    $mayor = $num1;
} elseif ($num2 > $num3) {
    $mayor = $num2;
} else {
    $mayor = $num3;
}

if ($num1 < $num2 && $num1 < $num3) {
    $menor = $num1;
} elseif ($num2 < $num3) {
    $menor = $num2;
} else {
    $menor = $num3;
}
```

Desglose: los datos llegan como **texto** por la URL, así que se convierten con `intval()`. La media se redondea con `round( ... , 2)`. El **mayor** y el **menor** se calculan comparando con `if/elseif` y los operadores lógicos `&&`.

Funciones usadas: `intval()`, `round()`. Operadores `>`, `<`, `&&`.

cadena.php — operaciones con cadenas

```php
$cadena1 = $_GET['cadena1'];
$cadena2 = $_GET['cadena2'];

$concatenada = $cadena1 . " " . $cadena2 . " del barrio";   // unir con .
$largo1 = mb_strlen($cadena1);                              // longitud (caracteres)
$largo2 = mb_strlen($cadena2);
$ultimos10 = mb_substr($cadena2, -10);                      // últimos 10
$reemplazada = str_replace("Paco", "Maria", $cadena1);      // sustituir
```

Desglose: el punto `.` **concatena**. `mb_strlen()` cuenta caracteres. En `mb_substr($cadena2, -10)`, el **-10** significa "desde el final" → los últimos 10. `str_replace("Paco","Maria", ... )` cambia "Paco" por "Maria".

Funciones usadas: `mb_strlen()`, `mb_substr()`, `str_replace()`.

infoServidor.php — datos del servidor

```
$nombreServidor   = $_SERVER['SERVER_NAME'];
$softwareServidor = $_SERVER['SERVER_SOFTWARE'];
$versionPhp       = PHP_VERSION;
```

Desglose: `$_SERVER` es una **superglobal** con datos del servidor. `PHP_VERSION` es una **constante** con la versión de PHP.

Qué resuelve: leer información del entorno de ejecución.

Superglobal usada: `$_SERVER`; constante `PHP_VERSION`.

4.5 UT3_1 — Estructuras de control

Ocho ejercicios (el 7 y el 8, obligatorios) mostrados en pestañas. Cada ejercicio es un fichero incluido con `include`.

Ejercicio 1 — Lanzar una moneda

```
$moneda = rand(0, 1);   // 0 o 1 al azar

if ($moneda == 0) {
    echo $cara;   // dibujo SVG de la cara
    echo "<p class='fs-4 mt-2'>Ha salido <strong>CARA</strong></p>";
} else {
    echo $cruz;   // dibujo SVG de la cruz
    echo "<p class='fs-4 mt-2'>Ha salido <strong>CRUZ</strong></p>";
}
```

Desglose: `rand(0,1)` simula el resultado. Como no hay imágenes, dibujo cara y cruz con **SVG en línea** (círculos, líneas y una sonrisa con `path`). El `if` decide qué dibujo mostrar.

Funciones usadas: `rand()`.

Ejercicio 2 — Nota aleatoria y su calificación

```php
$nota = rand(1, 10);

if ($nota < 5) {
    $categoria = "Insuficiente";  $color = "text-danger";
} elseif ($nota < 6) {
    $categoria = "Suficiente";    $color = "text-warning";
} elseif ($nota < 7) {
    $categoria = "Bien";          $color = "text-warning";
} elseif ($nota < 9) {
    $categoria = "Notable";       $color = "text-success";
} else {                          // 9 y 10
    $categoria = "Sobresaliente"; $color = "text-success";
}
```

Desglose: cadena `if/elseif` ordenada **de menor a mayor** (se comprueba `< 5`, luego `< 6` ...) para que los intervalos no se solapen. Se guarda también un **color de Bootstrap** según el tramo.

Qué resuelve: clasificar en tramos `[0,5) [5,6) [6,7) [7,9) [9,10]`.

Funciones usadas: `rand()`; condicionales.

Ejercicio 3 — ¿Dos o tres números iguales?

```php
$num1 = rand(1, 3);
$num2 = rand(1, 3);
$num3 = rand(1, 3);

if ($num1 == $num2 && $num2 == $num3) {
    $mensaje = "hay tres numeros iguales a $num1";
} elseif ($num1 == $num2 || $num1 == $num3 || $num2 == $num3) {
    if ($num1 == $num2 || $num1 == $num3) {
        $repetido = $num1;
    } else {
        $repetido = $num2;
    }
    $mensaje = "hay dos numeros iguales a $repetido";
} else {
    $mensaje = "no hay numeros iguales";
}

echo "<p class='fs-4'>" . ucfirst($mensaje) . ".</p>";
```

Desglose: el primer `if` detecta los **tres iguales** (`&&`). El `elseif` detecta que **al menos dos** coinciden (`||`). Dentro, se averigua **cuál** se repite. `ucfirst()` pone la primera letra en mayúscula.

Funciones usadas: `rand()` , `ucfirst()` ; operadores `&&` , `||` , `==` .

Ejercicio 4 — Suma de los pares anteriores a un número

Necesita **dos páginas**: el formulario (`ejercicio4.php`) y el resultado (`sumaPares.php`).

Formulario:

```
<form action="ejercicios/sumaPares.php" method="get" class="row g-2 align-items-center">
  <input type="number" id="numero" name="numero" min="0" class="form-control" required>
  <button type="submit" class="btn btn-primary">Calcular</button>
</form>
```

`sumaPares.php` :

```
$numero = intval($_GET['numero'] ?? 0);   // ?? = si no viene, usa 0

if ($numero ≤ 0) {
    echo "<div class='alert alert-warning'>No has introducido un numero positivo valido.
        </div>";
} else {
    $suma = 0;
    $pares = [];
    for ($i = 0; $i < $numero; $i += 2) {   // de 2 en 2 → solo pares
        $suma += $i;
        $pares[] = $i;
    }
    echo "<p>Numeros pares anteriores: " . implode(", ", $pares) . "</p>";
    echo "<div class='alert alert-success'>La suma ... es <strong>$suma</strong>.</div>";
}
```

Desglose: el formulario envía `numero` por **GET**. `$_GET['numero'] ?? 0` evita el error si se abre la página sin pasar por el formulario. El bucle `for` avanza `$i += 2` para pasar **solo por los pares** y usa `< $numero` (los **anteriores**). `implode(", ", $pares)` junta el array en un texto.

Qué resuelve: acumular valores en un bucle y validar la entrada.

Funciones usadas: `intval()` , `implode()` ; operador `??` .

Ejercicio 5 — Tabla Unicode del 0 al 50000

```php
for ($i = 0; $i ≤ 50000; $i++) {
    echo "&#$i; ";   // p. ej. &#241; = ñ
}
```

Desglose: un `for` que va del 0 al 50000 (`≤` para incluir el 50000). Se imprime `&#codigo;` generando un carácter por cada código. Se mete en un `<div>` con scroll para no romper la página.

Qué resuelve: generar muchos caracteres a partir de su código.

Funciones usadas: `for` (no hay función nativa; es el bucle).

Ejercicio 6 — Tabla de multiplicar de un número al azar

```php
$numero = rand(1, 12);

for ($i = 1; $i ≤ 10; $i++) {
    $resultado = $numero * $i;
    echo "<tr><td>$numero x $i</td><td><strong>$resultado</strong></td></tr>";
}
```

Desglose: se genera el número y se recorre del 1 al 10 pintando una fila de tabla por cada producto.

Funciones usadas: `rand()`; bucle `for`.

Ejercicio 7 (obligatorio) — Contar frutas

```php
$totalFrutas = rand(7, 20);

$frutas = [];
for ($i = 0; $i < $totalFrutas; $i++) {
    $frutas[] = rand(127815, 127827);   // códigos Unicode de frutas
}

$posicionElegida = array_rand($frutas);          // posición al azar
$frutaElegida = $frutas[$posicionElegida];

$veces = 0;
foreach ($frutas as $fruta) {                    // contar apariciones
    if ($fruta == $frutaElegida) {
        $veces++;
    }
}
```

Y luego:

```
<p><?= ($veces == 1 ? 'vez' : 'veces') ?></p>   <!— singular/plural con ternario —>
```

Desglose: `rand(7,20)` decide cuántas frutas. Se **rellenan** con `$frutas[] = ...` (cada `[]` añade al final). `array_rand()` devuelve una **clave al azar**. El `foreach` recorre para **contar** cuántas veces sale la fruta elegida. El **ternario** elige "vez"/"veces".

Qué resuelve: crear un array dinámicamente, elegir al azar y contar.

Funciones usadas: `rand()`, `array_rand()`; `foreach`, ternario.

Ejercicio 8 (obligatorio) — Tablas

Apartado 1: cuadrícula de números del 1 al 100 (10×10).

```
for ($i = 1; $i ≤ 100; $i++) {
    if ($i % 10 == 1) { echo "<tr>"; }   // abre fila en 1, 11, 21…
    echo "<td>$i</td>";
    if ($i % 10 == 0) { echo "</tr>"; }  // cierra fila en 10, 20, 30…
}
```

Desglose: el operador **módulo** `%` abre fila cuando el resto es 1 y la cierra cuando es 0, formando la cuadrícula sin bucles anidados.

Apartado 2: cuadrícula de emoticonos al azar.

```
$emoticonos = [128169, 128168, 127866, 128405, 129313];

for ($fila = 1; $fila ≤ 10; $fila++) {
    echo "<tr>";
    for ($columna = 1; $columna ≤ 10; $columna++) {
        $posicion = array_rand($emoticonos);
        $codigo = $emoticonos[$posicion];
        echo "<td class='fs-4'>&#$codigo;</td>";
    }
    echo "</tr>";
}
```

Desglose: aquí sí hay **bucles anidados** (10×10). En cada celda se elige un emoticono al azar del array con `array_rand()`.

Funciones usadas: `array_rand()`; operador `%`; bucles anidados.

4.6 UT3 — Arrays: Tareas 1 a 4

Tarea 1 — Arrays de películas (foreach y colores)

```php
$peliculas = [
    'Gladiator', 'El irlandés', 'Inception',
    'El Padrino', 'Donnie Darko', 'Shutter Island',
];

// 1) Imprimir cada película
foreach ($peliculas as $pelicula) {
    echo "<p>Película: $pelicula</p>\n";
}

// 2) Con la posición (empezando en 1)
foreach ($peliculas as $posicion => $pelicula) {
    echo "<p>Película " . ($posicion + 1) . ": $pelicula</p>\n";
}

// 3) Tabla con un color aleatorio por película
foreach ($peliculas as $posicion => $pelicula) {
    $r = random_int(0, 255);
    $g = random_int(0, 255);
    $b = random_int(0, 255);
    $numero = $posicion + 1;
    echo "<tr><td>$numero</td><td style=\"color: rgb($r, $g, $b)\">$pelicula</td></tr>\n";
}
```

Desglose: `foreach` sin clave recorre **valores**; con `$posicion => $pelicula` recorre **clave y valor**. La posición **empieza en 0**, por eso se le suma 1. `random_int(0,255)` genera el rojo/verde/azul y se aplica con `rgb( … )`. Se usa el **heredoc** `<<<HTML … HTML` para volcar un bloque de HTML/CSS sin comillas.

Funciones usadas: `random_int()`; `foreach`.

Tarea 2 — Bucles `for` (del 1 al 10, 60 al 70, 20 al 1, 1 al 1000, tabla del 5, tabla cruzada)

```php
for ($i = 1;  $i ≤ 10;  $i++) { echo "$i<br>"; }     // ascendente
for ($i = 60; $i ≤ 70;  $i++) { echo "$i<br>"; }     // rango
for ($i = 20; $i ≥ 1;   $i--) { echo "$i<br>"; }     // descendente
for ($i = 1;  $i ≤ 1000;$i++) { echo "$i<br>"; }     // mucho rango

for ($i = 0; $i ≤ 10; $i++) {                          // tabla del 5
    echo "5 x $i = " . (5 * $i) . "<br>";
}

for ($i = 0; $i ≤ 10; $i++) {                          // tabla cruzada
    for ($j = 0; $j ≤ 10; $j++) {
        echo "$i x $j = " . ($i * $j) . "<br>";
    }
}
```

Desglose: se practica el `for` en todas las direcciones: ascendente, rango concreto y **descendente** (`$i--`). El último ejemplo usa **bucles anidados** donde cada celda es el producto de los dos contadores.

Funciones usadas: bucles `for`.

Tarea 3 — Acumulador y bucles anidados

```php
$suma = 0;
for ($i = 1; $i ≤ 100; $i++) {
    $suma += $i;               // acumular
}
echo "La suma de los números del 1 al 100 es: $suma<br>";

$frutas = ['fresa', 'naranja', 'uva'];
$colores = ['rojo', 'verde'];

$vueltasExterior = 0;
$vueltasInterior = 0;
$echos = 0;

foreach ($frutas as $fruta) {
    $vueltasExterior++;
    foreach ($colores as $color) {
        $vueltasInterior++;
        $echos++;
        echo "La $fruta es $color<br>";
    }
}
echo "Primer foreach: $vueltasExterior veces<br>";
echo "Segundo foreach: $vueltasInterior veces<br>";
echo "echo realizados: $echos<br>";
```

Desglose: `$suma += $i` es un **acumulador**. Los bucles anidados recorren "combinaciones": 3 frutas × 2 colores = 6 `echo`. Los contadores sirven para **comprobar** cuántas veces entra cada bucle.

Funciones usadas: acumuladores; `foreach` anidado.

Tarea 4 — Formulario con `select` (día, mes, año) y `checkdate()`

```php
$anioActual  = (int) date('Y');
$diaElegido  = $_POST['dia'] ?? '';
$mesElegido  = $_POST['mes'] ?? '';
$anioElegido = $_POST['anio'] ?? '';
```

```
<form method="post">
  <select name="dia" id="dia">
    <?php
      $dias = range(1, 31);
      foreach ($dias as $dia) {
          $sel = ($dia == $diaElegido) ? ' selected' : '';
          echo "<option value=\"$dia\"$sel>$dia</option>";
      }
    ?>
  </select>
  <!— mes: for de 1 a 12 · año: while de 1900 a $anioActual —>
</form>
```

```
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    if ($diaElegido !== '' && $mesElegido !== '' && $anioElegido !== '') {
        if (checkdate((int) $mesElegido, (int) $diaElegido, (int) $anioElegido)) {
            echo "<p>Has nacido el $diaElegido/$mesElegido/$anioElegido</p>";
        } else {
            echo "<p>Esa fecha no existe</p>";
        }
    } else {
        echo "<p>Elige los tres campos</p>";
    }
}
```

Desglose: el formulario usa `method="post"`, así que los datos llegan por `$_POST`. Los desplegables se **rellenan con bucles**: `range(1,31)` + `foreach` para los días, `for` para los meses y `while` para los años (de 1900 al actual). El **ternario** `($x == $y) ? ' selected' : ''` recuerda la opción elegida. `$_SERVER['REQUEST_METHOD']` comprueba que la petición sea POST, y `checkdate()` valida que la fecha exista.

Qué resuelve: rellenar `select` dinámicamente y validar una fecha real.

Funciones usadas: `range()`, `date()`, `checkdate()`; superglobales `$_POST`, `$_SERVER`; bucle `while`.

4.7 Retos extra (material complementario)

4.7.1 Matrices — “CU00824B” (academia)

Ejemplo de **matriz** (array de arrays) para guardar alumnos y sus notas:

```php
$alumnos = [
    ["Alumno" => "Pepe",   "Nota" => 7],
    ["Alumno" => "María",  "Nota" => 9],
];

foreach ($alumnos as $alumno) {
    echo $alumno["Alumno"] . " tiene un " . $alumno["Nota"] . "<br>";
}
```

Desglose: cada elemento de `$alumnos` es a su vez un **array asociativo**. El `foreach` recorre las filas y dentro se accede a sus claves.

4.7.2 Arrays (andros.dev) — spread, diccionarios, cadenas como array

```php
$base = ['a', 'b'];
$todo = ['inicio', ...$base, 'fin'];   // spread: ['inicio','a','b','fin']

$dic = ['uno' => 1, 'dos' => 2];       // "diccionario" = array asociativo

$texto = 'Hola';
echo $texto[0];                        // H  (una cadena es indexable)
```

Desglose: el operador **spread** `...` expande un array dentro de otro. En PHP los "diccionarios" son **arrays asociativos**. Y una **cadena** se puede indexar como un array.

Funciones/operadores usados: operador `...`; acceso por índice.

Bloque 5 — Glosario general

Término	Definición
Aplicación web	Programa que el cliente usa a través del navegador y que se ejecuta sobre un servidor.
Apache	Servidor web más usado con PHP; forma parte de XAMPP.
Array	Variable que almacena varios valores bajo un mismo nombre.
Array asociativo	Array cuyas claves son cadenas en vez de números.
Back-end	Parte de la aplicación que se ejecuta en el servidor.
Bootstrap	Framework CSS para maquetar rápido (clases como `container`, `card`, `nav-tabs`).
Callback	Función que se pasa como argumento a otra función.
CDN	Red de servidores que sirve recursos (CSS/JS) desde la nube.
Cliente	Programa que realiza peticiones (el navegador).
Constante	Valor que no cambia, definido con `define()`, usado sin `$`.
CGI	Interfaz para ejecutar programas externos desde el servidor.
`date()`	Función que devuelve la fecha/hora formateada.
`echo`	Construcción del lenguaje que imprime texto/valores.
Excepción	Error controlable (`try` / `catch` / `throw`).
Expresión	Cualquier fragmento de código que produce un valor.
`foreach`	Bucle para recorrer arrays.
Front-end	Parte de la aplicación que se ejecuta en el navegador.
GET	Método HTTP que envía datos en la URL.
`header()`	Función para enviar cabeceras HTTP (p. ej. redirecciones).
`include` / `require`	Insertar el contenido de otro fichero.
`intval()`	Convertir un valor a entero.
`mb_*`	Funciones multibyte que cuentan caracteres (no bytes).
Matriz	Array multidimensional.
`null`	Valor de una variable sin valor.
Operador ternario	`cond ? a : b`, if/else abreviado.

Término	Definición
POST	Método HTTP que envía datos en el cuerpo de la petición.
Regex	Expresión regular: patrón para buscar/validar texto.
`round()`	Redondear a un número de decimales.
Sanitizar	Limpiar/escapar datos del usuario antes de usarlos.
Scope (ámbito)	Zona donde una variable existe.
Servidor	Programa que atiende peticiones y genera respuestas.
Superglobal	Array predefinido accesible desde cualquier punto (`$_GET`, `$_POST`, `$_SERVER`…).
`switch`	Condicional múltiple sobre el valor de una variable.
XAMPP	Paquete Apache + MySQL/MariaDB + PHP + Perl.

Anexo A — Resumen de funciones

Cadenas

Función	Devuelve
`strlen($s)`	Longitud en bytes
`mb_strlen($s)`	Longitud en caracteres
`strtoupper` / `strtolower`	Mayúsculas / minúsculas
`mb_strtoupper` / `mb_strtolower`	Ídem (multibyte)
`ucfirst($s)`	Primera letra en mayúscula
`ucwords($s)`	Inicial de cada palabra en mayúscula
`substr($s,$ini,$len)`	Trozo de cadena
`mb_substr($s,$ini,$len)`	Trozo de cadena (caracteres); `-n` = desde el final
`str_replace($b,$p,$s)`	Sustituir todas las apariciones
`str_contains($s,$sub)`	¿Contiene? (PHP 8)
`strpos($s,$sub)`	Posición (o `false`)
`substr_count($s,$sub)`	N.º de apariciones
`trim` / `ltrim` / `rtrim`	Quitar espacios
`explode($sep,$s)`	Cadena → array
`implode($sep,$arr)`	Array → cadena
`htmlspecialchars($s)`	Escapar HTML

Números

Función	Devuelve
`abs($x)`	Valor absoluto
`round($x,$n)`	Redondeo a `$n` decimales
`floor` / `ceil`	Suelo / techo
`max` / `min`	Mayor / menor
`rand($min,$max)`	Aleatorio
`random_int($min,$max)`	Aleatorio seguro
`intval` / `floatval`	Conversión a número

Tipos y estado

`is_int`, `is_float`, `is_bool`, `is_string`, `is_array`, `is_null`, `is_numeric`, `isset`, `empty`, `settype`, `var_dump`, `print_r`.

Fechas

`date`, `time`, `mktime`, `strtotime`, `checkdate`.

Arrays

`count`, `range`, `array_rand`, `array_merge`, `sort`, `rsort`, `in_array`, `array_keys`, `array_values`, `array_push`, `array_pop`.

Anexo B — Errores frecuentes y buenas prácticas

Errores frecuentes

1. **Olvidar el `;`** al final de una sentencia → error de sintaxis.
2. **Usar comillas simples pensando que interpolan:** `'Hola $nombre'` muestra literal `$nombre`. Con comillas dobles sí interpola.
3. **`strlen` con tildes/ñ:** cuenta bytes → usar `mb_strlen`.
4. **Comparar con `==` números que llegan de un formulario:** llegan como texto; convertir con `intval()` o usar `==`.
5. **No inicializar variables:** genera `E_WARNING` / `E_NOTICE`. Inicializar (`$suma = 0;`) o usar `?? 0`.
6. **`switch` sin `break`:** provoca *fall-through* (ejecuta casos de más).
7. **Condiciones mal ordenadas en `if/elseif`:** si pones primero la condición más amplia, las demás nunca se alcanzan.
8. **Rutas de `include`:** son relativas al fichero **que se ejecuta**, no al que incluye.
9. **Cerrar `?>` en ficheros solo-PHP:** puede inyectar espacios en blanco. Mejor no cerrar.
10. **Confiar en datos del usuario:** siempre validar (`filter_input`) y escapar (`htmlspecialchars`).

Buenas prácticas

- Usa `mb_*` siempre que trabajes con acentos y `ñ`.
- Inicializa las variables y evita avisos.
- Indenta y comenta el código (como en los ejercicios entregados).
- Separa **lógica** de **presentación**: la lógica en PHP, la vista en HTML.
- Cierra el código PHP donde toca y valida siempre la salida.
- En desarrollo, muestra errores (`display_errors=On`); en producción, regístralos y ocúltalos.

Apuntes elaborados a partir del material de las UT1, UT2 y UT3 del módulo DWES, los enunciados de ejercicios y las soluciones programadas para 2º DAW.